

Yep! 🚀 We've got the **cache invalidation mental model** down. Let's move to the next piece.

Next.js Caching — Next Part

🔥 Server Action vs Route Handler: Complete Under-the-Hood Flow

Now we'll connect everything we've learned into **one realistic mutation** and trace exactly what happens.

We'll use:

Show more ▾

Absolutely, Kanna 🚀 Let's continue exactly from where we stopped.

Lesson 5.8 — Server Action vs Route Handler

The complete mutation flow under the hood 🧠

The key thing I want you to understand first is:

Both Server Actions and Route Handlers can perform the same database mutation. The big difference is how Next.js coordinates the mutation with the UI and caches afterward.

We'll use one concrete example throughout.

1. Our application

Imagine:

/products



which displays products from a database.

For example:

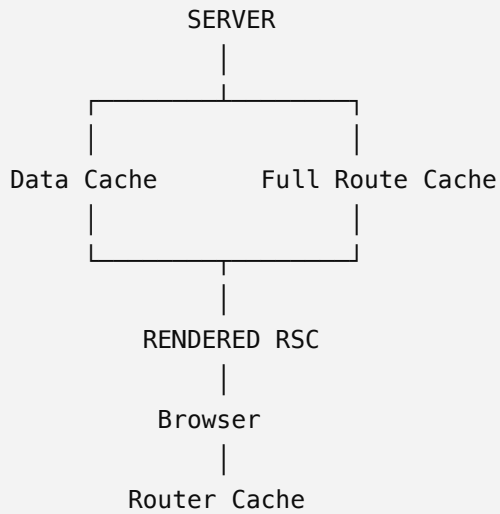
Products



iPhone	₹80,000
--------	---------

MacBook	₹1,20,000
AirPods	₹20,000

Our caching picture is:



Remember the responsibilities:

Data Cache

Stores **results of data fetching**.

```
fetch("/api/products")
  ↓
Data Cache
```

Full Route Cache

Stores the **rendered result of a route**:

```
/products
  ↓
HTML + RSC Payload
```

Router Cache

Lives in the **browser**.

It stores RSC payloads for routes the user has visited/prefetched.

```
Browser
  ↓
```

Router Cache



RSC Payload

2. Now let's perform a mutation

Suppose we have:

/products



and the user clicks:

Delete AirPods



The database currently contains:

iPhone
MacBook
AirPods



After mutation:

iPhone
MacBook



Now here's the important question:

How does the browser know that AirPods disappeared?

That's where Server Actions vs Route Handlers become interesting.

3. First: Server Action

Suppose we have:

```
"use server";

export async function deleteProduct(id: string) {
  await db.product.delete({
    where: { id }
  });
}
```

```
    revalidatePath("/products");  
  }
```

And our Client Component does:

```
<form action={deleteProduct}>  
  <button>Delete</button>  
</form>
```

The user clicks the button.

Let's follow the request **all the way through the system**.

4. Step 1 — Browser invokes Server Action

The browser sends a request to Next.js.

Conceptually:

```
graph TD  
  Browser -- "Server Action request" --> NextjsServer[Next.js Server]
```

This isn't simply:

```
POST /api/delete
```

in the traditional API-route sense.

Next.js knows:

"This request is invoking a particular Server Action."

5. Step 2 — Server Action executes

The server runs:

</> TypeScript



Database becomes:

Before:



iPhone
MacBook
AirPods

After:

iPhone
MacBook

So the **actual source of truth** has changed.

But...

⚠️ **The caches don't magically know what happened to the database.**

That's why we need invalidation.

6. Step 3 — `revalidatePath("/products")`

We explicitly say:

```
revalidatePath("/products");
```

Conceptually:

Mutation



Database changed



`revalidatePath("/products")`



`/products` becomes stale/invalidated



And because the Full Route Cache depends on the underlying data/rendering, the route's cached result needs to be regenerated when appropriate.

So mentally:

```
/products
```

```
↓
```

```
Full Route Cache
```

```
↓
```

```
INVALIDATED
```



And if the route's data came from cached data, that relevant data cache may also need to be invalidated depending on how the data is cached/tagged and how the revalidation API is being used.

This is an important nuance:

Don't memorize `revalidatePath = "delete exactly these cache entries"`.

Instead think:

"Make this path's cached result stale so the next render gets fresh data."

7. But here's the REALLY interesting part 🤪

What about the browser's:

```
Router Cache
```



Remember:

```
Router Cache
```

```
↓
```

```
RSC Payload for /products
```



The browser may currently have:

```
/products
```



```
RSC Payload:
```

```
  iPhone
```

```
  MacBook
```

```
  AirPods
```

But the database now says:

```
iPhone
```

```
MacBook
```



We don't want the user staring at:

AirPods



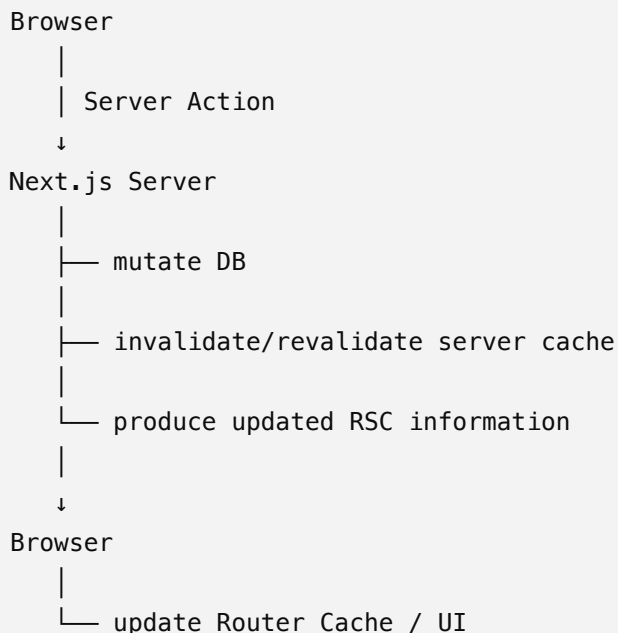
forever.

This is where Server Actions have a major advantage.

8. Server Action coordinates the UI

A Server Action can participate in the Next.js App Router lifecycle.

Conceptually:



So the action isn't merely:

"Hey server, delete this row."



It can effectively become:

"Delete this row,
invalidate the relevant server-side data,
and give the client the information needed
to reconcile the current App Router UI."



 **That's the magic you're trying to understand.**

9. What does the browser actually receive?

This is where we need to correct one common misconception.

The browser does **not necessarily receive a brand-new complete HTML document** after every Server Action.

Instead, the App Router works heavily with:

RSC Payload



The server can send the updated React Server Component result.

Conceptually:

Server



/products



render



RSC Payload



Browser

The browser then uses that RSC information to update the React tree.

So you can think:

Server Action



Server mutation



cache invalidation/revalidation



new server-rendered RSC result



browser receives RSC information



React updates UI

No full-page browser reload required.

10. Why does this feel so smooth?

Imagine you're on:

/products



You delete:

AirPods



You don't want:

DELETE

↓

full page reload

↓

download entire HTML

↓

reinitialize everything



Instead:

DELETE

↓

Server Action

↓

server updates data

↓

Next.js re-renders affected Server Components

↓

RSC Payload

↓

browser merges the result

↓

UI updates



That's why Server Actions can feel almost like:

local mutation



even though the database operation happened on the server.

11. Now let's introduce Route Handler

A Route Handler looks like:

```
// app/api/products/route.ts

export async function DELETE(request: Request) {
  await db.product.delete(...);

  return Response.json({
    success: true
  });
}
```

Now the browser calls:

DELETE /api/products



Flow:

```
Browser
  |
  | DELETE /api/products
  ↓
Route Handler
  |
  ↓
Database
  |
  ↓
Response
```



That's a normal HTTP API interaction.

12. Here's the critical difference

Suppose we're currently on:

/products



The browser's Router Cache has:

/products



iPhone
MacBook
AirPods

The Route Handler executes:

DELETE AirPods



Database becomes:

iPhone
MacBook



But the browser receives:

```
{  
  "success": true  
}
```

That's it.

The browser doesn't automatically say:

"Oh! `/products` is now stale. Let me ask Next.js for a new RSC tree and update the current route."

Because you called an HTTP API.

You have to coordinate that yourself.

13. So you might do this

Client Component:

```
const deleteProduct = async (id: string) => {  
  await fetch("/api/products", {  
    method: "DELETE",  
    body: JSON.stringify({ id })  
  });  
  
  router.refresh();  
};
```

Now:

```
fetch()  
↓
```



```

Route Handler
  ↓
Database mutation
  ↓
response
  ↓
router.refresh()
  ↓
request fresh RSC payload
  ↓
Server Components render again
  ↓
UI updates

```

🔥 That's the key difference.

14. `router.refresh()` vs `refresh()`

These two names are confusing as hell at first. 😞

They're **not the same thing**.

`router.refresh()`

Used on the **client**:

```

"use client";

import { useRouter } from "next/navigation";

const router = useRouter();

router.refresh();

```

It basically says:

"Refresh the current route's Server Component data/rendering."

It requests fresh RSC information while preserving the client-side navigation context.

Think:

```

CLIENT
  |
  | router.refresh()
  ↓
Get fresh RSC

```



↓
Update current UI

15. refresh()

This one is a Server Action API:

```
"use server";

import { refresh } from "next/cache";

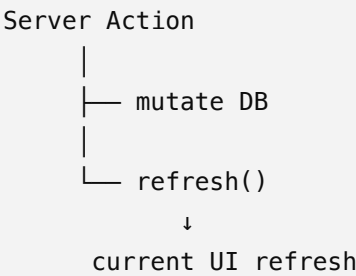
export async function deleteProduct() {
  await db.product.delete(...);

  refresh();
}
```

Conceptually:

"After this Server Action finishes, tell the client to refresh the current route."

So:



16. Very important distinction

Think of them like this:

API	Where?	Meaning
router.refresh()	Client	Client asks to refresh current route
refresh()	Server Action	Server Action tells the current client to refresh
revalidatePath()	Server	Invalidate/revalidate path's cached result

API	Where?	Meaning
<code>revalidateTag()</code>	Server	Invalidate data associated with tag
<code>updateTag()</code>	Server	Expire tagged data immediately / read-your-writes pattern

So these aren't interchangeable.

17. `revalidatePath()` vs `refresh()`

This distinction is **very important for interviews**.

Imagine:

User deletes product



`revalidatePath`

You're saying:

"The cached result associated with this path is stale."

`revalidatePath("/products")`



This is primarily about **cache invalidation/revalidation**.

`refresh()`

You're saying:

"Refresh the current route in the user's UI."

`refresh();`

This is about **triggering the current client's Router/UI refresh from a Server Action**.

So:

`revalidatePath()`
↓
cache freshness



```
refresh()  
  ↓  
current UI refresh
```

They can absolutely be used together:

```
"use server";  
  
export async function deleteProduct(id: string) {  
  await db.product.delete({  
    where: { id }  
  });  
  
  revalidatePath("/products");  
  refresh();  
}
```

Although whether you need both depends on your exact caching/mutation setup.

18. Complete Server Action timeline

Let's put everything together.

User clicks:

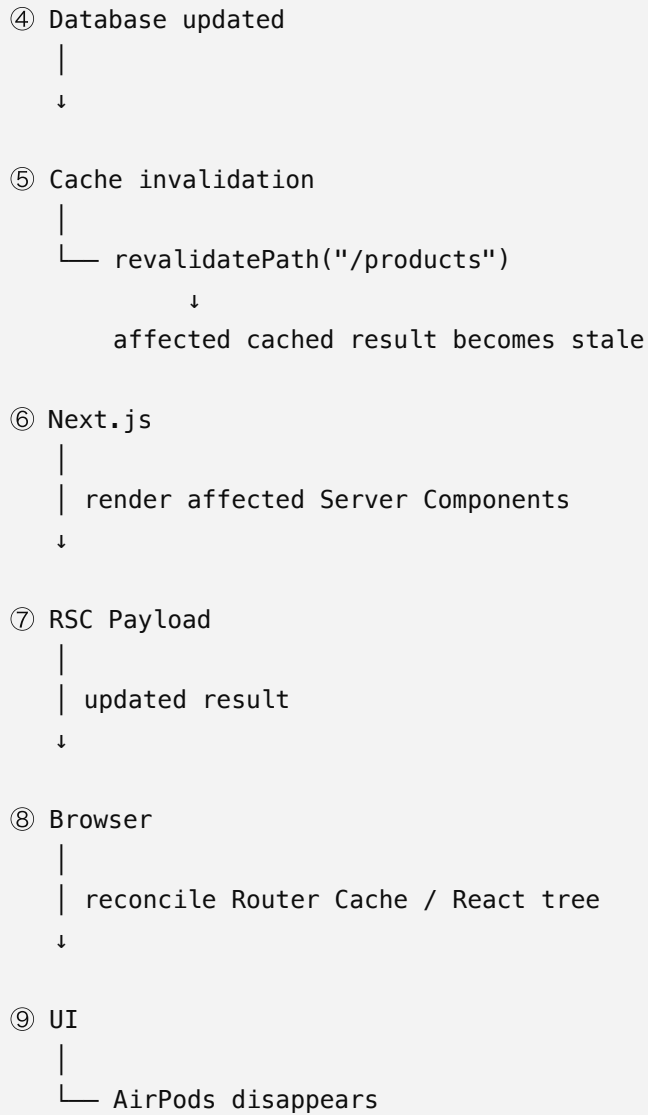
Delete AirPods



Timeline

```
① Browser  
  |  
  | invoke Server Action  
  ↓  
  
② Next.js Server  
  |  
  | execute action  
  ↓  
  
③ Database  
  |  
  | DELETE AirPods  
  ↓
```





The user never had to manually:

```
fetch()
router.refresh()
```



That's the beautiful part.

19. Route Handler timeline

Now the same operation through an API.





Why?

Because your API response:

```
{
  "success": true
}
```

doesn't contain an updated RSC tree.

So you might do:

```
await fetch("/api/products", {
  method: "DELETE"
});

router.refresh();
```

Then:

```
Route Handler
  ↓
DB mutation
  ↓
HTTP response
  ↓
router.refresh()
  ↓
new RSC Payload
```



↓
UI update

20. One subtle but VERY important point

Don't think:

"Route Handlers cannot invalidate caches."

That's false.

A Route Handler can use server-side cache APIs where supported.

For example, conceptually:

```
export async function DELETE() {  
  await db.product.delete(...);  
  
  revalidatePath("/products");  
  
  return Response.json({ success: true });  
}
```

So:

Route Handler
↓
can invalidate server cache



The important difference is:

The Route Handler itself doesn't automatically have the same current-UI coordination that a Server Action invocation provides.

If your browser separately called:

/api/products



Next.js doesn't inherently know:

"This browser is currently looking at /products , and after this API mutation it should reconcile that page."

You generally need:

```
router.refresh()
```

or another explicit client-side update strategy.

21. Why Server Actions are so nice for mutations

Compare them.

Server Action

```
<form action={deleteProduct}>
```



or:

```
await deleteProduct(id);
```

Conceptually:

```
UI
↓
Server Action
↓
Mutation
↓
Cache coordination
↓
RSC update
↓
UI
```



Route Handler

```
fetch("/api/products", {
  method: "DELETE"
})
```



Conceptually:

```
UI
↓
HTTP request
```



↓
Route Handler
↓
Mutation
↓
HTTP response
↓
YOU decide how UI updates

That distinction is gold. 🏆

22. Why would we still use Route Handlers?

Because they're **APIs**.

Suppose your backend needs to be consumed by:

```
Next.js web app
├── mobile app
├── third-party client
├── external service
└── another frontend
```

A Route Handler gives you:

```
GET  /api/products
POST /api/products
PUT  /api/products/:id
DELETE /api/products/:id
```

That's a standard HTTP interface.

Server Actions aren't primarily designed to be your public REST API.

23. Where do "API Routes" fit?

This terminology causes confusion.

There are two major concepts:

Old Pages Router API Routes

```
pages/api/products.ts
```



These belong to the **Pages Router**.

Modern App Router Route Handlers

```
app/api/products/route.ts
```



These are the modern equivalent for defining HTTP endpoints in the App Router.

So when people casually say:

"Next.js API routes"

they might mean either:

```
pages/api/*
```



or:

```
app/**/route.ts
```

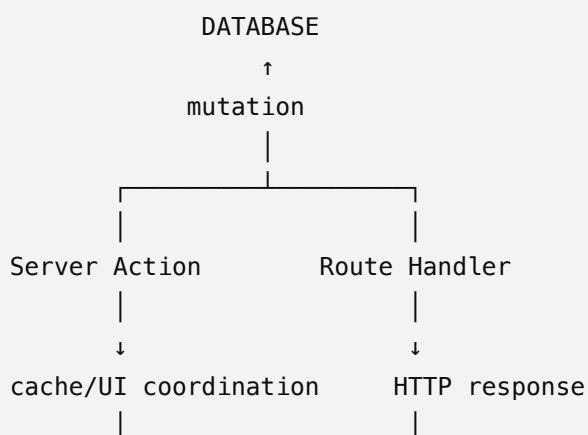


You need to know which one they're talking about.

24. The mental model I want you to remember

Don't memorize 50 Next.js rules.

Remember these three layers:



↓
RSC/UI update

↓
YOU coordinate UI

Or even simpler:

Server Action = mutation + Next.js-aware UI coordination

Route Handler = HTTP API endpoint

That's the foundation.

25. Interview question 🎯

If interviewer asks:

"What's the difference between a Server Action and a Route Handler?"

A strong answer is:

A Server Action is a server-side function intended to be invoked from the UI, especially for mutations. It integrates closely with the App Router and can coordinate cache invalidation and the current UI/RSC state. A Route Handler defines a standard HTTP endpoint, such as GET, POST, PUT, or DELETE. It's useful when you need an API boundary for clients or external consumers, but the client-side UI refresh/update usually needs to be handled explicitly.

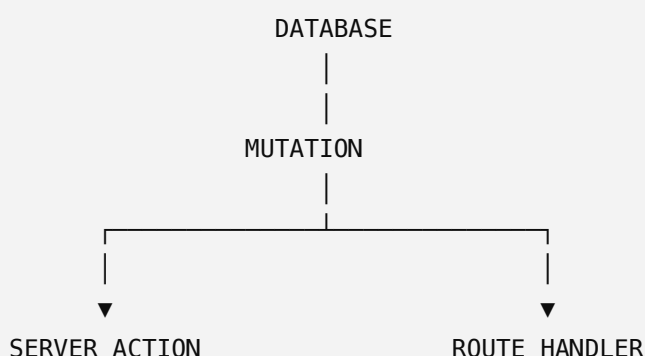
That's a **much better answer** than:

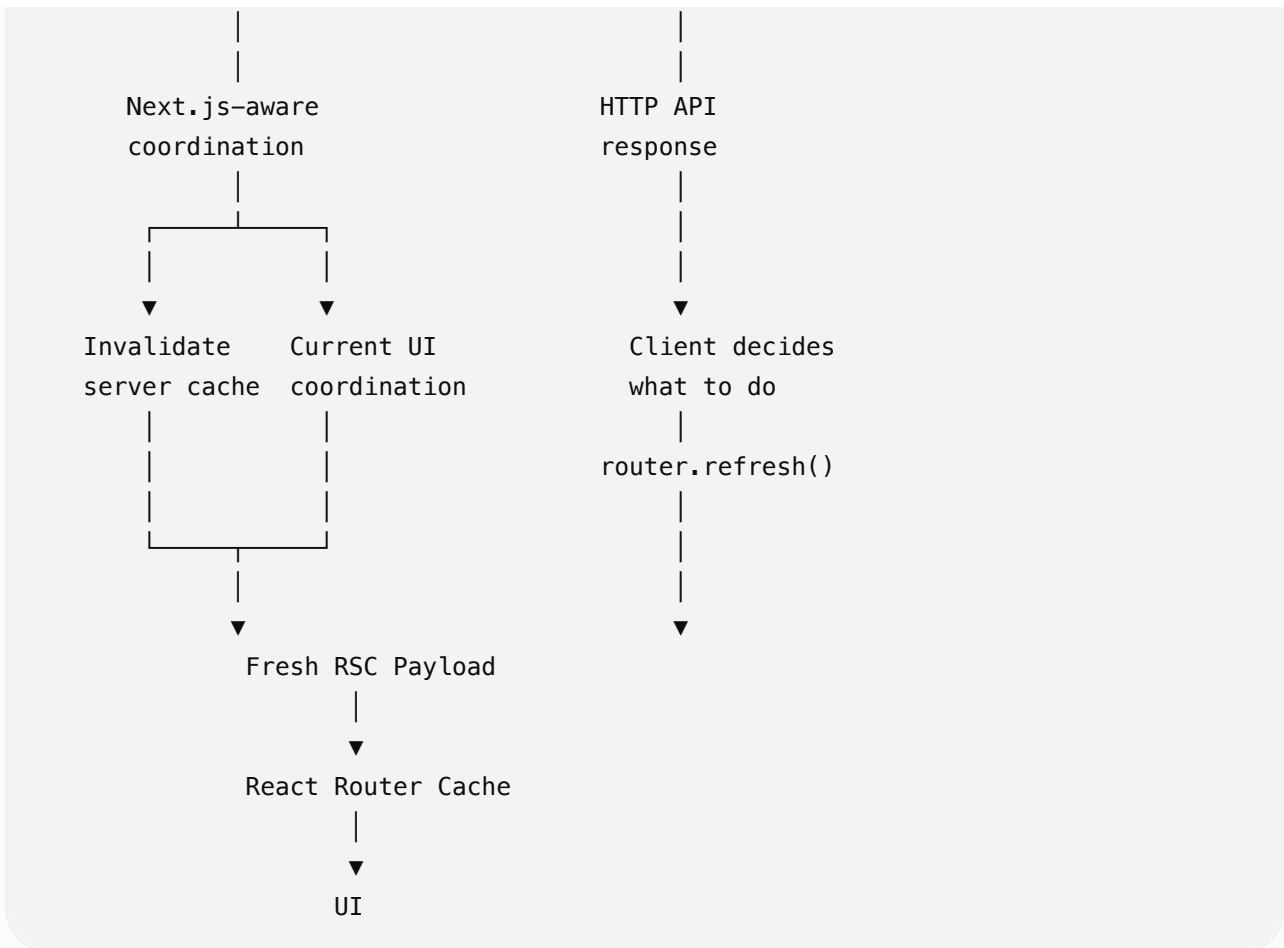
"Server Actions are for mutations and Route Handlers are APIs."

The second is technically directionally correct, but the first shows you understand **what happens under the hood**. 🔥

26. One final picture

Here's the whole thing we've built:





And now we are ready for the next natural piece:

👉 Lesson 5.9 — App Router vs Pages Router

We'll answer:

- Why Next.js has **two routing systems**
- `app/` vs `pages/`
- `page.tsx` vs `pages/*.tsx`
- Server Components vs traditional React rendering
- Layouts
- Nested routes
- Loading/error boundaries
- Data fetching differences
- API Routes vs Route Handlers
- `getServerSideProps`
- `getStaticProps`
- `getStaticPaths`
- How all of this connects to **SSG / SSR / ISR**
- And finally, why the **App Router** was introduced in the first place

That will bridge us directly into your **SSG → SSR → ISR → rendering under the hood** section. 🧠🔥



ChatGPT can make mistakes. Check important info.